MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL
(A constituent unit of MAHE, Manipal)

Mini Project Report

Of

Cloud Computing and DevOps Lab [CSE 3130]

WARDEN

A Verifiable Kubernetes Delivery Platform

SUBMITTED BY

Student	Registration number
Abhyuday Gupta	240905576
Aditya Chetan Thakur	240905588
Sankalp Arya	240905578
Shaurya Jain	240905598

Under the Guidance of

Mohandas Shenoy

School of Computer Engineering

Manipal Institute of Technology, Manipal

Academic Year 2026–27

MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL
(A constituent unit of MAHE, Manipal)

CERTIFICATE

This is to certify that the mini project report entitled “Warden: A Verifiable Kubernetes Delivery Platform” is submitted by Abhyuday Gupta (240905576), Aditya Chetan Thakur (240905588), Sankalp Arya (240905578), and Shaurya Jain (240905598) for the Cloud Computing and DevOps Lab [CSE 3130], School of Computer Engineering, Manipal Institute of Technology, during the academic year 2026–27.

The report describes the work and results presented by the team. The signatures below are reserved for examination and approval.

Project guide	**Examiner**
Signature: _____	Signature: _____
Mohandas Shenoy	Name: _____
Date: _____	Date: _____

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION

CHAPTER 2: METHODOLOGY

CHAPTER 3: RESULT ANALYSIS

CHAPTER 4: CONCLUSION

REFERENCES

Figures and tables are numbered by chapter. Evidence files and public release records are listed with the relevant results.

CHAPTER 1: INTRODUCTION

1.1 Context

A software delivery platform should make a release easy to produce and easy to verify. In Kubernetes, those are separate problems. A build can pass while a deployment uses a mutable image tag, a different artifact, or a release from an unexpected workflow. Warden connects the build record to the workload that the cluster receives. The project treats the delivery platform as the product; the four sample services exist to exercise it.

The platform covers service creation, build and scanning, image signing, GitOps promotion, admission checks, progressive rollout, and a declarative PostgreSQL request. It runs locally on kind and was also rebuilt and measured on Azure Kubernetes Service (AKS). The same Git repositories describe both environments. [1], [2]

1.2 Problem and objectives

A valid image signature alone does not establish that the expected repository, branch, or workflow produced the image. Warden therefore checks the artifact digest and its build identity, while separating the pipeline from cluster credentials. The project set six measurable objectives:

Objective	Experiment
Create and deploy a service from one command	E1
Refuse artifacts that fail admission checks	E2
Abort a faulty canary without a manual rollback	E3
Provide a PostgreSQL database from a short request	E4
Measure delivery from deployment history	E5
Rebuild the environment from versioned configuration	E6

Table 1.1. Objectives and their evaluation experiments.

1.3 Cloud services and scope

AKS supplies managed Kubernetes; GitHub Actions runs the build; GitHub Container Registry stores immutable images; Microsoft Entra ID and GitHub OpenID Connect provide workload identity; Sigstore issues signing certificates and records transparency entries; Azure Storage holds encrypted OpenTofu state. Argo CD applies the GitOps repository, Argo Rollouts controls canary traffic, and Crossplane composes a CloudNativePG database. These services demonstrate compute, registry, identity, storage, infrastructure as code, and delivery automation. [1], [3]

The intended security outcome remains partly staged. Digest, registry, and pod hygiene rules enforce in the live cluster. Cryptographic image verification remains in Audit because transient retrieval errors and verification timeouts can reject legitimate Pods. A Deny-mode copy was tested separately. The promised managed Azure PostgreSQL composition is also unfinished; both environments used CloudNativePG. Those limits are included in the results, rather than counted as completed objectives. [4]

CHAPTER 2: METHODOLOGY

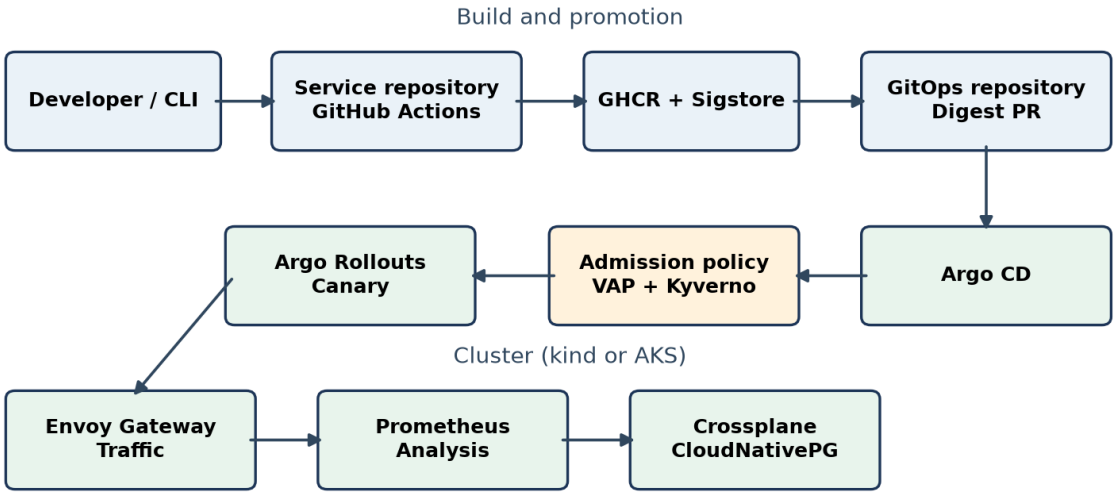


Figure 2.1. Warden delivery and cluster control flow.

2.1 Workspace and infrastructure

The project is split into a hub and four platform repositories: infrastructure, GitOps, policy, and platform tooling. OpenTofu prepares Azure resources. For local trials, a kind cluster replaces AKS while the GitOps and admission paths stay comparable. Pinned component versions and image digests keep rebuilds repeatable. Cluster creation and destruction were timed from a clean state, with failures retained in the rebuild ledger. [1], [5]

2.2 Service creation and trusted build

The Warden command creates a service repository, a small application, a reusable delivery workflow, deployment manifests, and a catalog entry. A push calls the shared platform workflow. It builds and scans the image, signs the digest, attaches provenance, and opens a narrow GitOps pull request that changes the deployed digest. The signing job alone receives an identity token. No delivery job receives cluster credentials; Argo CD later pulls the merged change. Third-party workflow actions are pinned to full commit hashes. [1], [3]

2.3 Promotion, admission, and rollout

A required digest-only check constrains each promotion pull request to one image digest. The promoter application can merge after that check passes. Argo CD applies the desired state. Native admission rules reject mutable tags, unapproved registries, and unsafe Pod settings. Kyverno verifies signatures and provenance against the intended build identity in Audit mode. For the controlled attack experiment, the same policy was exercised in Deny mode. Argo Rollouts then sends a small traffic share to the new version and uses Prometheus metrics to promote or abort it. Envoy Gateway supplies the route. [3], [4], [6]

2.4 Database and measurement

A namespaced database request is reconciled by Crossplane and CloudNativePG. The composition creates a predictable connection Secret, allowing the application manifest to remain the same across kind and AKS. Database readiness and a client connection were recorded. Delivery metrics use GitHub deployment events rather than a transient cluster log. Attack outcomes, canary timing, rebuilds, and verification latency were recorded as JSON or CSV under the evidence directory. [5], [7]

CHAPTER 3: RESULT ANALYSIS

3.1 Experiment summary

ID	Measured result	Interpretation
E1	3 full kind runs; median 8.6 min; 0 manual steps	One-command local path shown; Azure and planned sample size open.
E2	15/15 controlled attack cases on kind and AKS	Deny-mode test copy refused attacks; live image verification still audits.
E3	10/10 faulty canaries aborted; 10/10 healthy promoted	Median abort 147.2 s; faulty request share median 0.40%.
E4	Database ready: kind 804 s cold, 21 s warm; AKS 138 s	Same request and Secret shape; both backed by CloudNativePG.
E5	20 releases, 4 injected faults	Median lead time 12.6 min; failure and rework 20%; recovery 12.6 min.
E6	Local 11/12 rebuilds; Azure 5 successful rebuilds	Azure success median 22 min, with 4 failed attempts documented.
E7	One run each against idpbuilder	Warden used 8.8 GiB; idpbuilder 2.2 GiB and 8.5 min to first app.
E8	Warm verification 14–15 s per Pod	Similar on kind and AKS; cost is inside the verification path.

Table 3.1. Recorded results; the sample and scope are stated for each experiment. [5], [7]–[10]

3.2 Canary behavior

The controlled rollout series contained ten deliberately faulty and ten healthy releases. Every faulty release aborted, and every healthy release promoted. The median time from rollout start to abort was 147.2 seconds; the healthy promotion median was 471.6 seconds. The error trials exposed a median 0.397% of requests to the faulty canary, below the planned 0.5% share. The run data supports the rollback claim for kind; no comparable Azure canary series was run. [8]

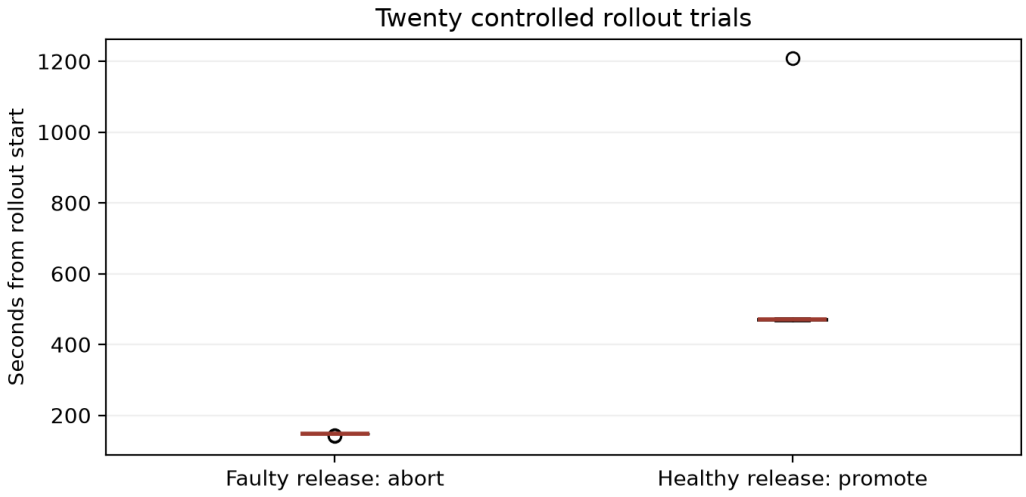


Figure 3.1. Time to abort or promote across 20 controlled kind rollouts; one healthy run took 1,209.5 s. Source: evidence/rollbacks/summary-2026-10-25T09.json.

3.3 Delivery and rebuild evidence

The E5 series used an automated schedule with one injected fault in every five releases. Its 20% change failure rate reflects that design, not a naturally occurring production rate. All E5 promotion pull requests were merged through an owner bypass after the digest-only check, so the measured lead time contains no human review delay. A later v0.1.2 release exercised application-only promotion in all four services: the build, scan, sign, and promote jobs succeeded, and the promoter authored and merged GitOps pull requests #97–#100. This proves the newer merge path works but does not turn the older E5 series into an application-only trial. [9], [11]

Service	GitOps pull request	Author and merger
notify	#97	warden-ccd-promoter
loans	#98	warden-ccd-promoter
members	#99	warden-ccd-promoter
books	#100	warden-ccd-promoter

Table 3.2. Verified v0.1.2 digest promotions on 27 October 2026. [11]

Local rebuilds succeeded in 11 of 12 recorded runs, with a median of 14.9 minutes and a 24.2-minute 90th percentile. Five Azure rebuilds succeeded from an empty state with a median of 22 minutes. Four failed Azure attempts remain in the ledger and were analyzed rather than discarded. [5]

3.4 Admission and database limits

The controlled Deny-mode policy suite admitted a legitimate workload and rejected A01–A14 for their expected reasons on kind and AKS. This is evidence for the rule logic, but it does not establish fail-closed behavior of the installed Audit policy. Verification took 14–15 seconds for a warm Pod, and transient registry or TUF fetch errors sometimes interrupted legitimate checks. The live cryptographic rule therefore remains in Audit pending an upstream correction and a new readiness run. [4], [7]

The database request worked in both environments and generated a usable connection URI. It did not demonstrate a managed PostgreSQL backend on Azure; that composition was not built. The result establishes portability of the request and Secret contract across clusters, not portability across database providers. [7]

CHAPTER 4: CONCLUSION

Warden demonstrates a connected path from service creation to a measured Kubernetes rollout. It creates services without manual deployment steps in the local trials, builds immutable images through a shared workflow, records signatures and provenance, promotes digest changes through GitOps, and rolls back faulty canaries under measured traffic. The system was rebuilt repeatedly on kind and AKS, and its delivery figures come from recorded releases rather than estimates.

Two promised outcomes need more work before they can be described as complete. The installed cryptographic admission policy still audits instead of denying, even though a controlled Deny-mode suite passed. The Azure database experiment used CloudNativePG rather than Azure Database for PostgreSQL. The E1 sample is also smaller than planned, and the 20-release E5 batch predates the verified application-only promotion fix. These boundaries matter because they distinguish a successful test setup from an enforced production path.

The next engineering step is to remeasure image verification after a Kyverno fix, including legitimate Pods, registry failures, and timeout behavior before changing Audit to Deny. A managed Azure PostgreSQL composition and its lifecycle test would complete the database objective. Repeating service creation and canary experiments on AKS, and rerunning the delivery series through the verified promoter path, would close the remaining evidence gaps. The present report preserves the results and limitations needed for that follow-up.

REFERENCES

[1] WARDEN-CCD, “Warden architecture and source,” GitHub, 2026. https://github.com/WARDEN-CCD/warden

[2] WARDEN-CCD, “Warden design and decisions,” DESIGN.md and DECISIONS.md, 2026. https://github.com/WARDEN-CCD/warden

[3] WARDEN-CCD, “Reusable delivery workflow and CLI,” warden-platform, 2026. https://github.com/WARDEN-CCD/warden-platform

[4] WARDEN-CCD, “Admission policy and threat model,” warden-policy and THREAT_MODEL.md, 2026. https://github.com/WARDEN-CCD/warden-policy

[5] WARDEN-CCD, “Rebuild ledger,” evidence/rebuilds/rebuild-ledger.csv, 2026. https://github.com/WARDEN-CCD/warden/blob/main/evidence/rebuilds/rebuild-ledger.csv

[6] WARDEN-CCD, “GitOps desired state and promotion check,” warden-gitops, 2026. https://github.com/WARDEN-CCD/warden-gitops

[7] WARDEN-CCD, “Evaluation procedures and evidence,” EVALUATION.md, 2026. https://github.com/WARDEN-CCD/warden/blob/main/EVALUATION.md

[8] WARDEN-CCD, “Controlled rollout summary,” 2026. https://github.com/WARDEN-CCD/warden/blob/main/evidence/rollbacks/summary-2026-10-25T09.json

[9] WARDEN-CCD, “E5 delivery metrics,” 2026. https://github.com/WARDEN-CCD/warden/blob/main/evidence/dora/2026-10-26-e5-kind.json

[10] WARDEN-CCD, “Verification cost and comparison evidence,” 2026. https://github.com/WARDEN-CCD/warden/tree/main/evidence

[11] WARDEN-CCD, “svc-books delivery run 40502485200 and GitOps pull request 100,” 2026. https://github.com/WARDEN-CCD/svc-books/actions/runs/40502485200 ; https://github.com/WARDEN-CCD/warden-gitops/pull/100